Poughkeepsie, N. Y.
October 2, 1961

To: J. Cocke
H. G. Kolsky

To bring you up-to-date on the Stretch book, we have (at last!) complete IBM approval. Meanwhile the Galleys have been pouring in. To save time, I am asking only one author of each chapter to proof-read it, and Dick Ballance has agreed to do this for Chap. 15.

On rereading that Chapter I find that the first paragraph (which I suspect I stuck in myself) really does not state the primary purpose of the look-ahead very well. So I am proposing to replace it by the 3 paragraphs attached. I am also enclosing the original for comparison. If you have any comments, please phone Dick or me immediately, as I am trying to get the Galleys into the mail to the publisher on Wednesday, if possible.

I don't think there are any other changes of substance.

W. Buchholz

WB:gs
attach.

cc: R. S. Ballance

(Material to replace first paragraph of Sec. 15.1)

The look-ahead unit is a speed-matching device interposed between the arithmetic unit and the memory. With multiple 2-μsec memory units--typically four units for data, each independently operable -- it is possible to fetch or store a data word every 0.5 μsec. This rate would be high enough to keep up with the fast arithmetic unit, as well as a number of input-output units, if it were not for unavoidable delays. The delay between the initiation of an operand transfer and the arrival of that operand at its destination is made up partly of the access time of the memory itself and partly of the time taken for the operand to pass through a series of switches and registers. More delay occurs if the desired memory unit happens to be busy finishing a previously initiated memory cycle or if it still needs to service a request with higher priority. The total waiting time may amount to several memory cycles.

The time spent by the arithmetic unit waiting for an operand may be greatly reduced by 'looking' several instructions ahead of the one currently being executed. If the memory reference is initiated early enough, the operand will usually be available in a buffer register by the time the arithmetic unit is ready for it. The look-ahead unit thus raises the effective speed of the arithmetic unit.

The look-ahead unit may also be considered as a buffer that helps to smooth the data flow through memory. With many parts of the system having independent access to memory, it is natural for peaks and valleys to occur in the demand for a given memory unit. Input-output units cannot be kept waiting long, and so they have a higher priority on memory than the central processing unit. If the CPU were simply stopped during a period of peak input-output activity, the waiting time would be lost completely. By having a reservoir for unexecuted instructions in the look-ahead registers, it is possible to make up some of the lost time by satisfying deferred CPU demand during a period of lower input-output activity. Thus the look-ahead helps to regulate the fluctuations in memory demand.

December 7, 1960

# Chapter 15 - The Look-ahead Unit *

R. S. Ballance, J. Cocke, and H. G. Kolsky

---

* The discussion of the results of a timing simulator study, which governed the choice of design parameters of the look-ahead unit, is taken from an earlier paper by J. Cocke and H. G. Kolsky, "The Virtual Memory in the Stretch Computer," Proceedings of the EJCC, no. 16 (December 1959); p. 82 - 93. The description of the simulator logic is omitted. That paper also described the look-ahead concept on which the simulator was based. This chapter includes instead a simplified description by R. S. Ballance of the actual look-ahead unit which was finally incorporated in the 7030.

---

## 15.1 General Description

The purpose of the look-ahead unit is to smooth the flow of data and instructions, and to act as a buffer between the fast arithmetic unit registers and the relatively slow multiple memory units. There is a considerable waiting time after the initiation of an operand transfer before the operand can reach its destination, partly because of the delay inherent in the memory itself, and partly because of the delays encountered in passing through a series of switches and registers. If the desired one of the several independently operable memory units happens to be busy finishing a previously initiated memory cycle or servicing a request of higher priority, there will be an additional delay until the unit is free. With many parts of the system having independent access to memory, it is natural for peaks and valleys to occur in the demands for a given memory unit. These fluctuations can be smoothed out considerably by looking several instructions ahead and initiating references to memory well before the data are needed for executing the instructions.